

《企业级应用软件设计与开发》

期末大作业报告

Mood Agent

基于 LangGraph 的心情驱动音乐歌单智能体

课程名称： 企业级应用软件设计与开发

项目名称： Mood Agent 心情歌单智能体

方 向： 方向一：Agentic AI 原生开发

学 号： 2025303043

姓 名： 许锦瀚

专 业： 软件工程

指导教师： 戚欣

提交日期： 2026 年 6 月 22 日

目 录

一、选题背景与设计思想.....	4
1.1 问题定义	4
1.2 现有方案的不足.....	4
1.3 项目价值	4
1.4 技术路线	4
二、Specs 规格文档.....	5
2.1 Product Spec（产品规格）	5
2.2 Architecture Spec（架构规格）	5
2.3 API Spec（接口规格）	6
三、系统架构与设计	6
3.1 总体架构	6
3.2 Agent 交互流程（状态机）	7
3.3 数据流设计	7
四、关键实现与代码展示.....	8
4.1 Agent 核心循环（LangGraph 编排）	8
4.2 记忆机制	8
4.3 工具定义与 MCP.....	9
4.4 配置与安全	9
4.5 AI IDE（Trae CN）使用	9
五、测试与评估	10
5.1 功能测试	10
5.2 Agent 行为评估（可观测性）	11
5.3 记忆机制验证.....	12
5.4 离线自测	12
六、系统升级与扩展	12
6.1 可扩展架构	12
6.2 下一阶段计划	13
6.3 AI 能力演进路径.....	13
七、课程总结.....	13

7.1 个人收获	13
7.2 工程思维的转变	13
7.3 对课程的建议	14
参考与致谢	14

一、选题背景与设计思想

1.1 问题定义

音乐已经成为学习、运动、通勤等场景中不可或缺的背景。然而主流音乐应用的歌单获取方式仍以“关键词搜索 + 静态推荐算法”为主，在真实使用中暴露出三个痛点：第一，无法理解自然语言所表达的情绪与场景，用户脑中“想要雨夜赶 due 时的专注背景乐”这样的诉求难以直接转化为检索词；第二，冷启动能力差且缺乏跨会话记忆，系统不记得用户的口味，每次都从零开始；第三，交互是单轮、无状态的，无法把“理解情绪 → 选曲 → 落地成歌单 → 记住偏好”这一过程串成可复用的闭环。

1.2 现有方案的不足

传统基于协同过滤或内容推荐的方案，依赖大量历史行为数据，对新用户与新场景不友好；同时它对自然语言的语义理解能力有限，难以捕捉细腻的情绪。更现实的一个约束来自外部 API 生态的变化：自 2024 年 11 月起，Spotify 对新注册应用关闭了 Recommendations、Audio Features、Audio Analysis、Related Artists 等一批核心端点，凡是直接依赖这些端点做推荐的实现都会失效。这促使本项目重新思考职责划分——把“理解情绪与推荐”交给大语言模型，把外部音乐服务仅用于检索与落地，从架构层面规避该限制。

1.3 项目价值

本项目以 Agentic AI 的方式重新组织歌单生成：用户用一句自然语言描述心情，由智能体自主完成“理解情绪 → 生成候选曲目 →（可选）解析并落地到 Spotify → 记住本次偏好”的完整闭环，并在后续为同一用户提供个性化推荐。它把大语言模型的语义理解能力、状态机的多步推理能力、外挂记忆的个性化能力以及工具调用能力组合在一起，体现了课程“从代码编写者到智能体编排者”的核心理念。

1.4 技术路线

系统以 Python 为主语言，采用 DeepSeek 作为大语言模型（OpenAI 兼容接口），以 LangGraph 构建有状态的智能体编排核心，使用 SQLite 实现跨会话记忆，通过 MCP 协议将音乐工具标准化对外暴露，由 FastAPI 提供服务接口，并以结构化日志实现节点级可观测性，整体以 Docker 容器化便于部署。开发过程采用 SDD（规格驱动开发）方法论，先定义 Product / Architecture / API 三份规格，再据此实现。

二、Specs 规格文档

本章给出本项目的三份核心规格，作为 SDD 规格驱动开发的依据。

2.1 Product Spec（产品规格）

目标用户为希望用一句话描述心情即可获得贴合歌单的普通听众，以及需要场景化背景乐（学习、运动、通勤等）的用户。核心功能如下表所示。

编号	功能	说明
F1	心情生成歌单	输入自然语言心情，输出歌单名与曲目列表
F2	跨会话记忆	记住同一用户历史心情与口味，个性化后续推荐
F3	可选落地 Spotify	配置后可将歌单写入真实 Spotify 账户
F4	工具化 / MCP	搜索与建歌单能力以标准 MCP 工具对外暴露

非目标：不提供音频流播放；不训练任何模型（同时规避 Spotify 关于使用其 API 数据训练模型的限制）。

2.2 Architecture Spec（架构规格）

系统分为四层：服务层（FastAPI）对外暴露 REST 接口；编排层（LangGraph Agent）以状态机形式串联多步推理；能力层包含 LLM、记忆、工具三类资源；协议层以 MCP Server 将工具标准化输出。可观测性作为横切关注点贯穿编排全过程。关键设计决策：其一，选用 LangGraph 是为了把“理解—生成—解析—落地—记忆”显式建模为有向图，天然支持条件分支与可观测，并便于扩展为多智能体；其二，外部音乐服务仅使用

search 与 create-playlist 端点，避免触碰已废弃端点；其三，在缺少 Spotify 凭据时自动降级，仅返回大模型歌单，保证核心闭环始终可演示。

2.3 API Spec (接口规格)

方法 / 路径	说明
GET /health	健康检查与能力探测 (llm_ready / spotify_enabled)
POST /generate	核心端点：按心情生成歌单，spotify_token 可选
GET /memory/{user_id}	查看某用户的历史记忆
GET /auth-url	返回 Spotify 授权 URL (需配置)
POST /token	用授权 code 换取 access token (需配置)

/generate 的返回中包含 personalized 字段：为 true 表示本次推荐参考了该用户的历史记忆；若配置并传入 spotify_token，则进一步填充曲目 uri 并返回真实歌单链接。

三、系统架构与设计

3.1 总体架构

系统总体架构如图 1 所示。用户通过浏览器 (Swagger UI 等) 访问 FastAPI 服务层，请求进入 LangGraph 编排核心，由其按状态机流程调度 DeepSeek、记忆库与音乐工具；MCP Server 将工具按协议标准对外暴露，可观测性模块对编排过程进行节点级追踪。

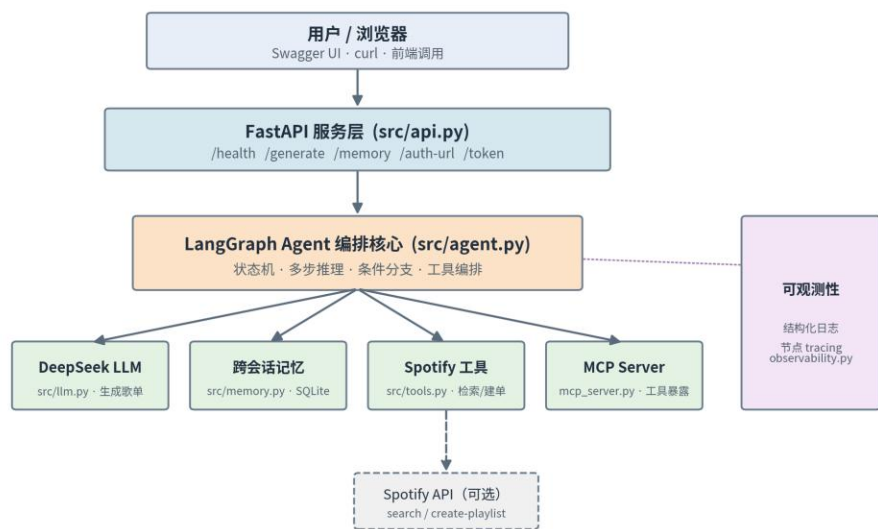


图 1 系统总体架构

图 1 系统总体架构

3.2 Agent 交互流程（状态机）

智能体的交互流程被建模为一个 LangGraph 状态机（图 2）：recall 读取用户记忆，compose 调用大模型生成歌单名与候选曲目，resolve 在具备 Spotify 凭据时把候选解析为真实曲目，随后由条件分支决定是否进入 publish 落地为歌单，最终统一在 remember 写入记忆。状态对象在节点间流转，体现了多步推理与显式状态管理。

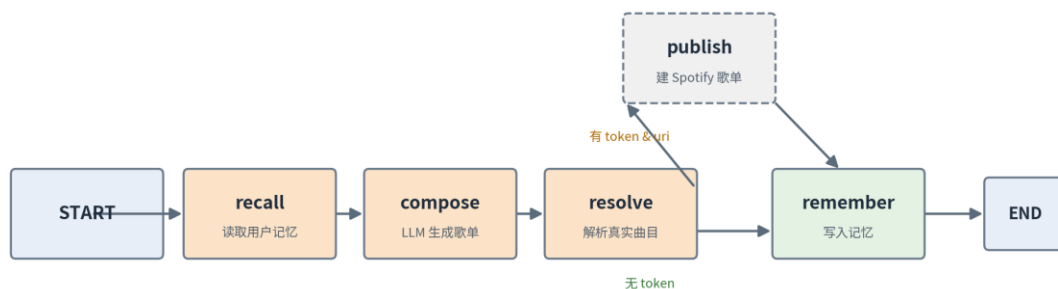


图 2 Agent 状态机（LangGraph 多步推理与条件分支）

图 2 Agent 状态机流程

3.3 数据流设计

一次请求的数据流为：心情文本与 user_id 进入状态对象 → recall 从 SQLite 取出历史偏好拼成上下文 → compose 将“心情 + 历史上下文”作为提示词交给 DeepSeek，得到结构化 JSON 歌单 → resolve 视情况调用 Spotify 检索补全 uri → 分支落地 → remember 将本次心情、歌单名与曲目持久化。历史上下文的回流使得同一用户的后续请求能够个性化，形成“存得下、取得回、用得上”的记忆闭环。

四、关键实现与代码展示

4.1 Agent 核心循环（LangGraph 编排）

核心循环由 build_graph 定义。节点以普通函数实现，返回对状态的增量更新；条件边根据是否具备 token 与 uri 决定走向 publish 或直接进入 remember。

```
def build_graph():
    g = StateGraph(MoodState)
    g.add_node("recall", recall)
    g.add_node("compose", compose)
    g.add_node("resolve", resolve)
    g.add_node("publish", publish)
    g.add_node("remember", remember)
    g.set_entry_point("recall")
    g.add_edge("recall", "compose")
    g.add_edge("compose", "resolve")
    g.add_conditional_edges("resolve", _route_after_resolve,
                           {"publish": "publish", "remember": "remember"})
    g.add_edge("publish", "remember")
    g.add_edge("remember", END)
    return g.compile()
```

4.2 记忆机制

compose 节点把从记忆库读取的历史上下文拼入提示词，实现个性化与去重；这是相对传统单轮生成的关键差异。

```
@obs.node
def compose(state):
    n = int(state.get("num_songs", 10))
    user = (f"心情描述: {state['mood_description']}\n"
            f"歌曲数量: {n}\n"
            f"{state.get('history_context', '')}\n\n"
            '请只返回 JSON: {"playlist_name": "...", '
            '"songs": [{"name": "...", "artist": "..."}]}')
```

```
data = llm.chat_json(SYSTEM_PROMPT, user)
return {"playlist_name": data.get("playlist_name", "Mood Mix"),
        "songs": data.get("songs", [])[:n]}
```

4.3 工具定义与 MCP

音乐工具（搜索、建歌单）既被状态机节点直接编排调用，也通过 MCP Server 按协议标准对外暴露，任何支持 MCP 的客户端均可发现并调用。

```
mcp = FastMCP("music-tools")

@mcp.tool()
def search_music(query: str, spotify_token: str, limit: int = 5) -> list:
    """在 Spotify 搜索曲目。"""
    return search_tracks(query, spotify_token, limit)

@mcp.tool()
def create_music_playlist(name, description, track_uris, spotify_token) -> str:
    """用给定曲目新建一个 Spotify 歌单，返回链接。"""
    return create_playlist(name, description, track_uris, spotify_token)
```

4.4 配置与安全

所有密钥均通过环境变量注入，代码中不硬编码任何 API Key；.env 已被 .gitignore 排除，不会进入版本库。系统提供能力开关，缺少 Spotify 凭据时自动降级。

```
class Settings:
    DEEPSEEK_API_KEY = os.getenv("DEEPSEEK_API_KEY", "")
    DEEPSEEK_BASE_URL = os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek.com")
    LLM_MODEL = os.getenv("LLM_MODEL", "deepseek-chat")

    @property
    def spotify_enabled(self):
        return bool(self.SPOTIFY_CLIENT_ID and self.SPOTIFY_CLIENT_SECRET)
```

4.5 AI IDE (Trae CN) 使用

本项目在课程指定的 Trae CN（AI 原生 IDE）中完成开发。下图为使用截图。

“解释一下 agent.py 里这个 LangGraph 状态机的流程”

SOLO Work

Read 1 file >

agent.py 中的 LangGraph 状态机是一个基于心情生成歌单的智能 Agent，它将整个流程拆分为 5 个节点 (node)，通过状态 (MoodState) 在节点间流转数据，并支持条件分支判断。下面是完整的流程解析：

1. 状态定义 (MoodState)

```
Python
1 class MoodState(TypedDict):
2     mood_description: str # 用户输入的心情描述
3     num_songs: int # 需要的歌曲数量
4     user_id: str # 用户ID
5     spotify_token: Optional[str] # Spotify 授权令牌 (可选)
6
7     # 中间/输出字段
8     history_context: str # 用户历史偏好
9     playlist_name: str # 生成的歌单名称
10    songs: List[dict] # 歌曲列表 (含歌名、歌手、URI等)
11    spotify_playlist_url: Optional[str] # 最终 Spotify 歌单链接
12    error: Optional[str] # 错误信息
```

所有节点共享同一个状态对象，每个节点读取所需字段，返回需要更新的字段。

2. 节点流程图

```
Plain Text
1 recall → compose → resolve ─┬─ (有 Spotify token & 解析到 URI) → publish → remember → END
2                               └─ (无 token 或 无 URI) → remember → END
```

图 2 Trae CN 使用

五、测试与评估

5.1 功能测试

通过 FastAPI 的交互式接口页面对核心端点进行功能测试。下表为代表性测试用例与结果。

用例	输入心情	预期	结果
健康检查	—	llm_ready 为 true	通过
生成歌单	雨夜赶 due 的专注背景乐	返回贴合的器乐/钢琴曲	通过
心情切换	开心的周末早晨	返回欢快流行乐，风格明显不同	通过
记忆个性化	同一用户二次请求	personalized 为 true	通过

图 3 为生成“雨夜专注”歌单的请求与返回，模型给出了 Gymnopédie No.1、River Flows in You 等贴合情绪的器乐作品。

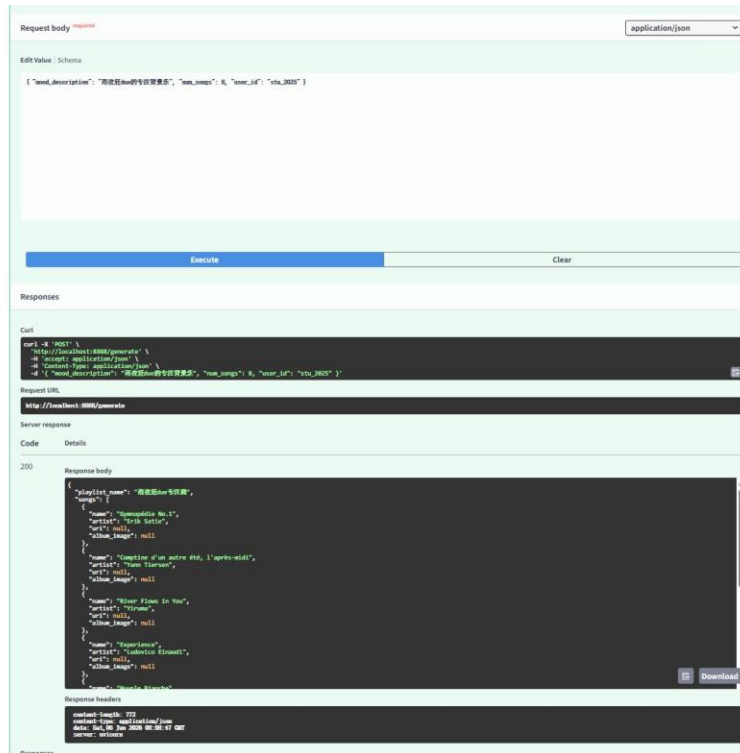


图 3 /generate 接口执行与返回（雨夜专注）

图 4 为切换到“开心的周末早晨”后的返回，曲目变为 Here Comes the Sun、Walking on Sunshine、Happy 等欢快流行乐，说明智能体确实在依据情绪生成，而非套用模板。



图 4 心情切换后的返回（欢快晨间）

5.2 Agent 行为评估（可观测性）

服务运行时输出节点级结构化日志，可清晰看到一次请求依次经过 recall → compose →

resolve → remember 各节点及其耗时（图 5），其中 compose 因调用大模型耗时最长。这为评估智能体的多步行为提供了直接依据。

```
INFO:      Unicorn running on http://127.0.0.1:8888 (Press CTRL+C to quit)
INFO:      Started reloader process [43128] using WatchFiles
INFO:      Started server process [48140]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      127.0.0.1:54476 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:50396 - "GET /docshttp%3A//localhost%3A8888/docs HTTP/1.1" 404 Not Found
INFO:      127.0.0.1:58853 - "GET /docs HTTP/1.1" 200 OK
INFO:      127.0.0.1:58853 - "GET /openapi.json HTTP/1.1" 200 OK
2026-06-06 16:06:08,139 | {"event": "agent_run", "user_id": "stu_2025", "mood": "雨夜赶due的专注背景音乐", "spotify": false}
2026-06-06 16:06:08,273 | {"event": "node_start", "node": "recall"}
2026-06-06 16:06:08,301 | {"event": "node_end", "node": "recall", "ms": 26.3, "updates": [{"history_context"}]}
2026-06-06 16:06:08,313 | {"event": "node_start", "node": "compose"}
2026-06-06 16:06:13,660 | {"event": "node_end", "node": "compose", "ms": 5347.3, "updates": [{"playlist_name", "songs"}]}
2026-06-06 16:06:13,661 | {"event": "node_start", "node": "resolve"}
2026-06-06 16:06:13,661 | {"event": "node_end", "node": "resolve", "ms": 0.0, "updates": []}
2026-06-06 16:06:13,661 | {"event": "node_start", "node": "remember"}
2026-06-06 16:06:13,687 | {"event": "node_end", "node": "remember", "ms": 25.8, "updates": []}
INFO:      127.0.0.1:61937 - "POST /generate HTTP/1.1" 200 OK
```

图 5 Agent 节点级运行日志（多步推理与耗时）

5.3 记忆机制验证

以同一 user_id（stu_2025）连续发起两次不同心情的请求后，第二次返回中 personalized 字段为 true，且 /memory 接口可查询到完整历史记录，证明记忆“存得下、取得回、用得上”。下面为第一次生成结果的节选（personalized 为 true）。

```
{
  "playlist_name": "雨夜赶 due 专注曲",
  "songs": [
    {"name": "Gymnopédie No.1", "artist": "Erik Satie"},
    {"name": "River Flows in You", "artist": "Yiruma"},
    {"name": "Nuvole Bianche", "artist": "Ludovico Einaudi"},
    {"name": "The Path of the Wind", "artist": "久石让"}
  ],
  "spotify_playlist_url": null,
  "personalized": true
}
```

5.4 离线自测

项目提供 selftest.py，以桩函数替换真实大模型调用，可在不消耗任何 API、不联网的条件下验证整条状态机的节点流转、条件分支与跨会话记忆读写，便于回归测试与答辩兜底。

六、系统升级与扩展

6.1 可扩展架构

当前以 LangGraph 状态机为骨架，新增能力只需新增节点或工具，对既有流程影响小。MCP 的引入使工具具备标准化对外能力，便于被其它智能体或客户端复用。

6.2 下一阶段计划

- 多智能体协作：拆分为选曲 (curator)、评审 (critic)、串场 (DJ) 三个智能体分工协作，提升歌单质量与连贯性。
- Agentic RAG：接入曲库向量检索，为大模型候选提供事实增强，缓解“幻觉曲目”。
- Spotify 落地完善：补全 OAuth 流程，将歌单一键写入真实账户并回填封面。
- 云端部署：基于已就绪的 Dockerfile 部署到云服务器，提供可公开访问的 URL。

6.3 AI 能力演进路径

短期以单智能体 + 记忆夯实核心体验；中期引入 Agentic RAG 与评估基准（如歌单相关性、情绪匹配度的人工/自动评测）形成 LLMOps 闭环；长期演进为多智能体协作与可插拔工具生态，使系统从“会生成”走向“会评估、会改进”。

七、课程总结

7.1 个人收获

通过本项目，我第一次完整地以智能体的方式组织一个软件系统：不再把大模型当作一次性的问答接口，而是将其嵌入到一个有状态、可观测、可扩展的编排流程中。我熟悉了 LangGraph 的状态机建模、Function Calling 与 MCP 的工具标准化、基于向量/数据库的记忆机制，以及用结构化日志评估智能体行为的方法。

7.2 工程思维的转变

最大的转变是从“写实现”到“做编排与规约”。先用 SDD 把产品、架构、接口规格定清楚，再据此实现，让系统边界更清晰；面对 Spotify 端点收紧这类外部不确定性，我学

会从职责划分上做架构级规避，并通过优雅降级保证系统在最小依赖下仍可运行——这些都是“智能体编排者”需要的工程素养。

7.3 对课程的建议

课程在“从代码编写者到智能体编排者”的定位非常贴合行业趋势。建议后续可补充更多关于智能体评估与 LLMOps 的实操内容（如自动化评测、成本与延迟监控），并提供一两个端到端的参考评测基准，帮助同学更系统地衡量智能体质量。

参考与致谢

本项目开发过程中参考了以下开源项目与文档，并在仓库 README 与 NOTICE 中按其许可证（Apache 2.0 等）作了标注：

- LangGraph / LangChain（智能体状态机编排框架）
- DeepSeek API 文档（OpenAI 兼容的大语言模型接口）
- Model Context Protocol (MCP) 官方 Python SDK

代码仓库：<https://github.com/Primacyyyyy/cs599-project>